EduOS Multi-Component Operating System Simulator Report

Module Information

Module Code: 351 CS 2104

Module Name: Operating Systems

Assignment: EduOS Multi-Component Simulator

Student Name: Samuel Francis

Registration Number: 25311351017

Submission Date: 28 may 2026

Abstract

EduOS is a multi-component operating system simulator developed using both the C programming language and Python. The project demonstrates major operating system concepts such as process management, threading, CPU scheduling, interprocess communication, synchronization, deadlocks, and system integration. The simulator was designed to imitate several low-level operating system operations while also visualizing scheduling algorithms using Python.

The C section of the project focused on low-level operating system functionality including Process Control Blocks (PCBs), thread pools, shared memory communication, pipes, synchronization using mutexes and semaphores, race conditions, and deadlock handling. The Python section implemented CPU scheduling algorithms such as FCFS, SJF, Priority Scheduling, and Round Robin while generating Gantt charts and performance metrics.

The project also integrated both languages using a controller program written in Python which communicates with the C simulator through JSON files and subprocess management. Through the development of EduOS, several important operating system concepts were explored practically rather than only theoretically.

Introduction

Operating systems are responsible for managing computer hardware resources and coordinating the execution of processes and threads. Understanding how operating systems work internally requires both theoretical and practical knowledge. EduOS was developed to simulate important operating system mechanisms using a combination of low-level C programming and high-level Python programming.

The project combines process management, CPU scheduling, thread synchronization, memory sharing, and interprocess communication into one integrated simulation environment. By implementing these concepts manually, the project provides a deeper understanding of how real operating systems manage resources and execute tasks.

The assignment also introduced the use of GitHub version control, collaborative software development practices, Makefiles, and software documentation.

Objectives

The objectives of this project were:

To simulate operating system process management using Process Control Blocks.

To implement threading and synchronization using POSIX pthreads.

To demonstrate race conditions and their solutions using mutexes.

To simulate interprocess communication using shared memory and pipes.

To implement CPU scheduling algorithms in Python.

To visualize scheduling behaviour using Gantt charts.

To integrate C and Python modules into one complete simulator.

To understand how real operating systems manage concurrency and resources.

To gain experience with GitHub version control and software project organization.

System Architecture

The EduOS simulator consists of three main components:

1. C Core Simulator

The C core handles low-level operating system functionality including:

Process creation

Process execution

Process termination

Thread management

Synchronization

Shared memory communication

Pipe communication

2. Python Scheduling Simulator

The Python module performs:

CPU scheduling simulations

Scheduling metrics calculations

Gantt chart generation

Algorithm comparison

3. Controller Layer

The controller integrates both systems together by:

Launching the C simulator

Reading PCB snapshot files

Passing data to the Python scheduler

Generating reports

Part 1 — GitHub Repository Setup

The project repository was created on GitHub using the required repository structure. The repository contained all required folders including documentation, screenshots, C source files, Python source files, and the integration controller.

The repository also included:

README.md

.gitignore

Makefile

requirements.txt

Screenshots

PDF report

Commit history

A minimum of ten meaningful commits were made throughout the project development process.

Part 2 — Core Simulator in C

Process Control Block (PCB)

A Process Control Block is a data structure used by operating systems to store information about active processes. In EduOS, each PCB contained:

Process ID

Process name

Process state

Priority

Burst time

Arrival time

Remaining time

Memory requirement

Thread count

Creation timestamp

The PCB structure was implemented in the header file eduos.h.

Example:

```
typedef struct {

pid_t pid;

char name[64];

int state;

int priority;

int burst_time;

int arrival_time;

int remaining_time;

int memory_req_kb;

int thread_count;

time_t creation_time;

} PCB;
```

Process Management Functions

edu_fork()

The edu_fork() function simulated the Linux fork() system call. It created a child process by copying the parent PCB and assigning a new process ID.

Functions performed:

Copy parent PCB

Generate new PID

Change process state

Save PCB snapshot

edu_exec()

The edu_exec() function simulated replacing the process image similar to Linux execve(). The function updated the program name and reset execution values.

edu_wait()

The edu_wait() function simulated a parent process waiting for child process termination.

edu_exit()

The edu_exit() function marked a process as terminated and recorded the exit code.

edu_ps()

The edu_ps() function displayed all active processes in a formatted table similar to the Linux ps command.

PCB JSON Serialization

Every time a process state changed, the PCB table was written into pcb_snapshot.json. This file allowed the Python scheduler to read process information.

Example JSON output:

[

{

"pid": 1000,

"name": "chrome.exe",

"state": 1,

"priority": 2

}

]

JSON serialization was implemented manually using fprintf() without external libraries.

Thread Pool and Thread Management

Thread management was implemented using POSIX pthreads. A thread pool was created using a fixed-size array of worker threads.

Important synchronization tools used:

pthread_mutex_t

pthread_cond_t

semaphores

The thread pool executed tasks concurrently while sharing resources safely.

Many-to-One Thread Model

The Many-to-One threading model maps many user threads onto one kernel thread. In EduOS, this model was simulated using cooperative execution.

Advantages:

Low overhead

Easy implementation

Disadvantages:

Blocking operations stop all threads

No true parallelism

One-to-One Thread Model

The One-to-One model maps every user thread directly to a POSIX thread.

Advantages:

True parallelism

Better responsiveness

Disadvantages:

Higher resource consumption

Increased context-switch overhead

Parallel summation tasks were used to demonstrate concurrency.

Race Condition Demonstration

A race condition occurs when multiple threads modify shared data simultaneously without synchronization.

In EduOS, multiple threads incremented a shared counter.

Without mutex protection:

counter++;

Results became non-deterministic because threads updated the value simultaneously.

After adding mutex protection:

pthread_mutex_lock(&lock;);

counter++;

pthread_mutex_unlock(&lock;);

The counter became correct consistently.

Producer Consumer Problem

The producer-consumer problem was implemented using semaphores.

Components:

Producer thread

Consumer thread

Shared buffer

Semaphore synchronization

Functions used:

sem_init()

sem_wait()

sem_post()

Deadlock Demonstration

A deadlock scenario was intentionally created using two mutexes acquired in opposite order.

Example:

Thread 1:

Lock Mutex A

Wait for Mutex B

Thread 2:

Lock Mutex B

Wait for Mutex A

The issue was solved using consistent lock ordering.

Interprocess Communication (IPC)

Shared Memory

Shared memory communication was implemented using:

shm_open()

mmap()

Two processes accessed a shared memory structure containing process metrics.

Advantages:

Fast communication

Direct memory access

Disadvantages:

Synchronization complexity

Mutexes were used to avoid simultaneous writes.

Anonymous Pipes

Anonymous pipes were implemented using:

pipe()

fork()

read()

write()

The parent process sent serialized PCB data to the child process.

The child process parsed and printed the received data.

Memory Management

Valgrind was used to detect memory leaks.

Command used:

valgrind --leak-check=full ./eduos

Memory allocation and deallocation were carefully handled to prevent leaks.

Makefile

The project Makefile included the following targets:

all

clean

race

fixed

memcheck

Example:

all:

gcc -Wall -Wextra -pthread -std=c11 *.c -o eduos

Part 3 — Scheduling Simulator in Python

Scheduling Algorithms

Four scheduling algorithms were implemented.

FCFS (First Come First Served)

FCFS executes processes according to arrival order.

Advantages:

Simple implementation

Fair arrival order

Disadvantages:

High waiting time

Convoy effect

SJF (Shortest Job First)

SJF executes the shortest burst time first.

Advantages:

Lowest average waiting time

Disadvantages:

Starvation risk for long processes

Priority Scheduling

Processes were executed according to priority values.

Lower numbers represented higher priority.

Ageing was implemented to prevent starvation.

Every three time units, waiting processes improved priority.

Round Robin Scheduling

Round Robin scheduling used a fixed time quantum.

Advantages:

Fair CPU sharing

Good responsiveness

Disadvantages:

High context-switch overhead

Different quantum values were tested to compare performance.

Scheduling Metrics

The simulator calculated:

Waiting Time (WT)

Turnaround Time (TAT)

Response Time (RT)

CPU Utilization

Throughput

These metrics were used to compare scheduling performance.

Gantt Charts

Matplotlib was used to generate Gantt charts.

The charts visually represented:

Process execution order

CPU idle periods

Scheduling behaviour

Charts were saved inside docs/screenshots/.

Comparison Charts

Comparison bar charts were generated to compare:

Average waiting time

Average turnaround time

I’ve created a complete EduOS assignment report covering:

Abstract

Introduction

Objectives

System architecture

C core implementation

PCB management

Threading and synchronization

IPC

Scheduling algorithms

Integration controller

OS concepts

Challenges and solutions

Lessons learned

Scheduling analysis

Conclusion

References

Commands used

SAMUEL FRANCIS 25311351017

Scheduling results